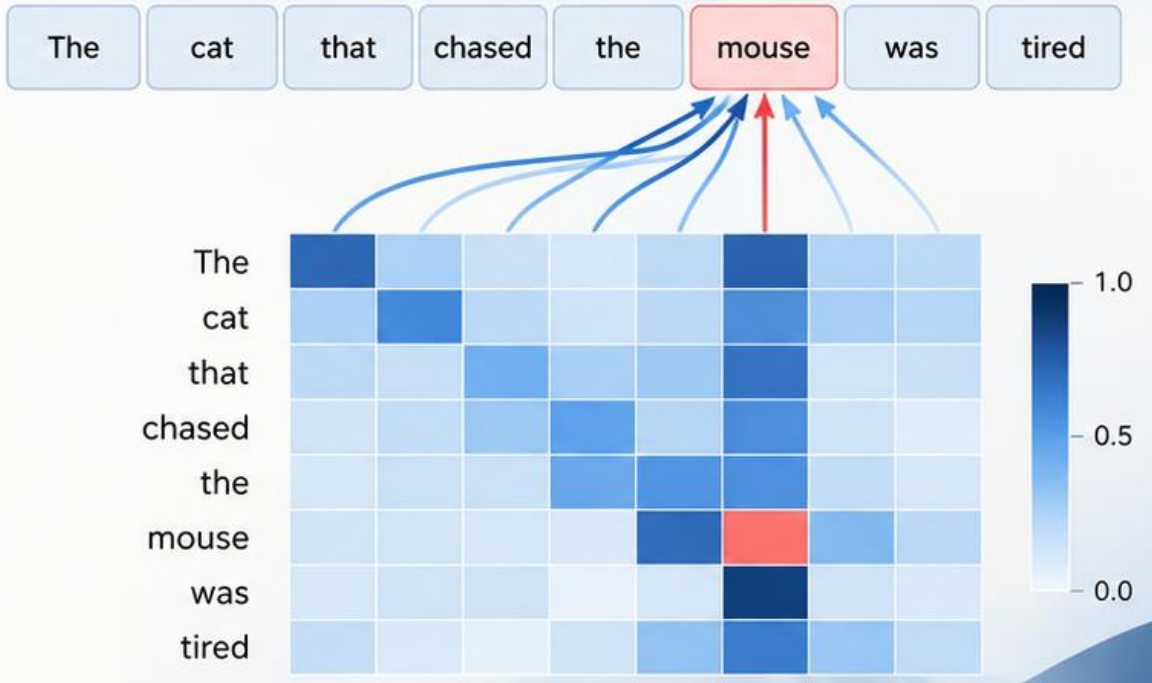
SIS 3204Y(5)
Natural Language Processing

Week 6

Attention Mechanisms for NLP

Attention Mechanism | Self-Attention | Multi-Head Attention



Learning Outcomes

By the end of this lecture, you should be able to:

1. 

Understand the motivation for attention
Explain the limitations of recurrent models and why attention mechanisms are needed in NLP.

2. 

Explain the core idea of attention
Describe how attention allows a model to focus on the most relevant information when making a prediction.

3. 

Work with Query, Key and Value
Explain the roles of queries, keys and values and how they are used to compute attention.

4. $$\alpha = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

Compute attention scores and weights
Calculate attention scores using the scaled dot-product attention formula and obtain attention weights using softmax.

5. 

Explain self-attention
Describe how self-attention computes relationships between tokens in the same sequence and why it is useful.

6. 

Understand multi-head attention
Explain how multiple attention heads work, what they capture and why they are beneficial.

7. 

Interpret attention weights
Analyse attention weight matrices and explain what they indicate about token relationships.

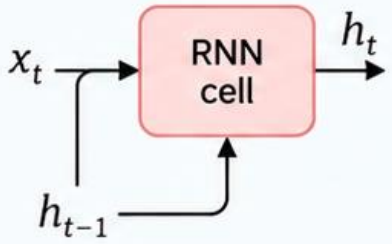
8. 

Apply attention to NLP tasks
Explain how attention and self-attention are used in tasks such as machine translation, text classification and sequence labelling.



Recap: From RNN to LSTM and GRU

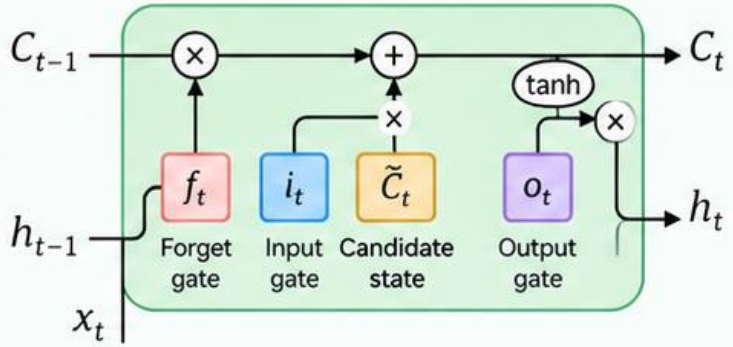
Recurrent Neural Network (RNN)



$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$
$$y_t = W_y h_t + b_y$$

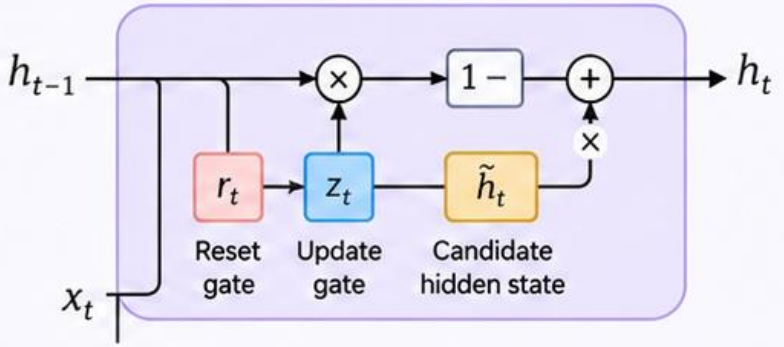
- Processes a sequence one step at a time
- Uses a hidden state h_t to carry information from previous steps
- Can model short-term dependencies
- Simple architecture with relatively few parameters

Long Short-Term Memory (LSTM)



- Introduces a cell state C_t to store information across many time steps
- Uses three gates: forget, input and output
- Can retain important information and remove unnecessary information
- Especially effective for long-term dependencies

Gated Recurrent Unit (GRU)



- Combines the cell state and hidden state into a single hidden state h_t
- Uses two gates: reset and update
- Simpler architecture than LSTM (fewer parameters)
- Performs well on many NLP tasks
- Also effective for modelling long-term dependencies

Key Limitation of Vanilla RNN

- Struggles with long-term dependencies
- Can suffer from vanishing (or exploding) gradients during training

How LSTM and GRU Address the Limitation

- Use gates to control the flow of information
- Retain important information over longer sequences
- Allow the model to selectively forget or update information
- More effective for capturing long-term dependencies in text

Summary

- RNN: simple, suitable for short sequences
- LSTM: gated memory with separate cell state
- GRU: simpler gated architecture
- Both LSTM and GRU improve the ability to capture long-term dependencies, which is important in NLP.

Why Do We Need Attention?

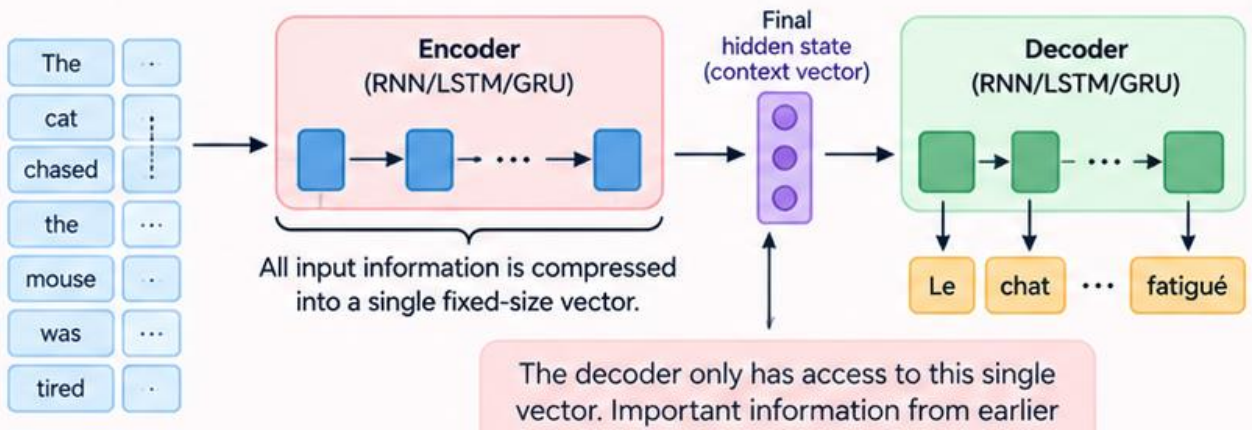
- Recurrent models (RNN, LSTM, GRU) process a sequence one word at a time and maintain a **hidden state**.
- For long sequences, the hidden state has to compress all previous information into a **fixed-size vector**.
- Important information from earlier words **can be lost** or become less accessible.
- **Not all words are equally important** when making a prediction — the model should focus on the most relevant words.
- Attention allows the model to **directly look at and selectively focus** on the relevant parts of the input sequence, instead of relying only on a single compressed representation.

Example: Machine Translation

The cat that chased the **mouse** was tired. → Le chat qui a poursuivi **la souris** était fatigué.

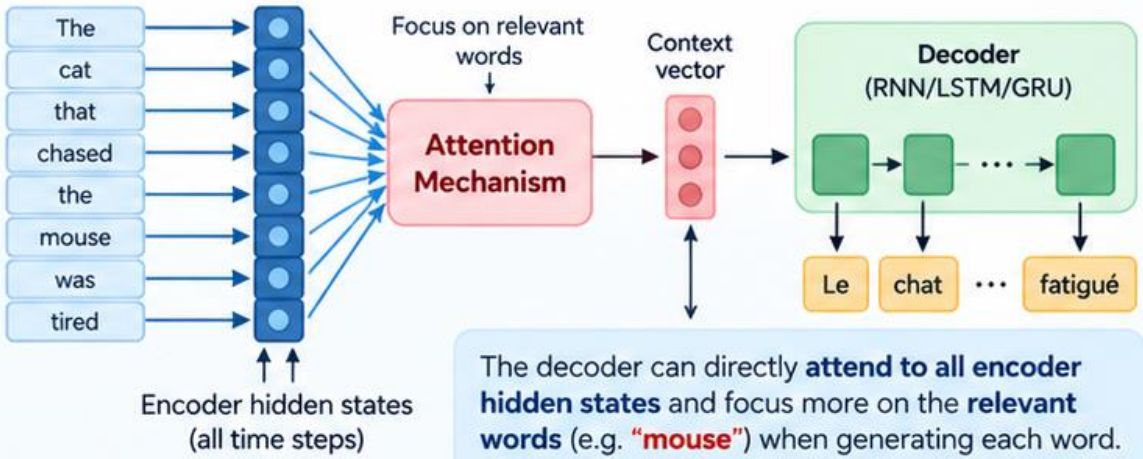
When generating the French word "**souris**", the model should focus more on **"mouse"** in the input rather than all the other words.

Encoder–Decoder WITHOUT Attention



The decoder only has access to this single vector. Important information from earlier words **can be lost**.

Encoder–Decoder WITH Attention



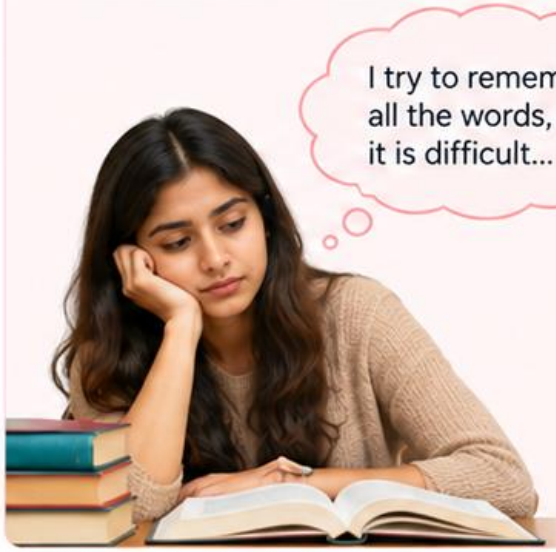
The decoder can directly **attend to all encoder hidden states** and focus more on the **relevant words** (e.g. **"mouse"**) when generating each word.

Attention — An Everyday Analogy

When we read a sentence, we do not treat all words equally. We naturally focus our attention on the most relevant words to understand the meaning.

Without Attention

Trying to remember everything at once

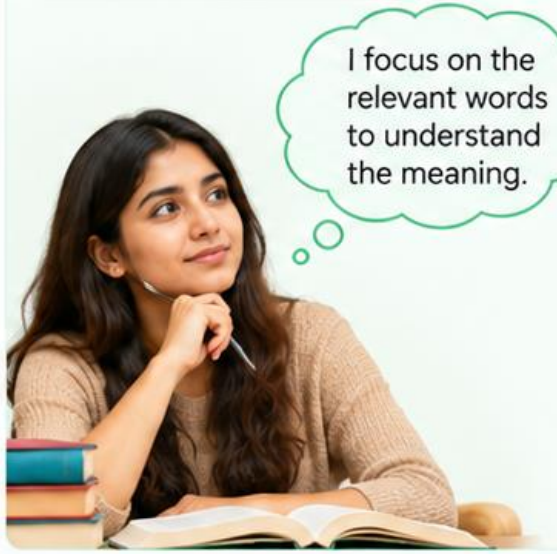


The cat that chased the mouse was tired because it ran all day in the garden.

- All words are given similar importance.
- It is difficult to remember and process the whole sentence at once.
- Important words may be lost in the long sequence.

With Attention

Focus on the most relevant information



The **cat** that chased the **mouse** was **tired** because it ran all day in the garden.

- Focus on the key words related to the current task.
- Relevant words receive more attention (e.g. “**mouse**” and “**tired**”).
- Helps to better understand the meaning and make the correct prediction.

Key Idea

Attention allows the model to focus on the most relevant parts of the input, rather than giving equal importance to all words. This helps the model to better understand context and make more accurate predictions.

Attention in NLP — Simple Example

When making a prediction, the model does not use all words in the input equally.
It assigns higher attention to the words that are more relevant to the current word or task.

Example: Word Meaning in Context

We want to understand the meaning of the word **"bank"** in the sentence:

	The	river	flooded	the	**bank**	after	heavy	rain	.
	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9
Attention weight (for w_5 'bank')	0.02	**0.35**	**0.20**	0.05	0.08	0.05	**0.15**	0.08	0.02

Higher attention on relevant words

Interpretation

- The model gives higher attention weights to **"river"** and **"flooded"** because they are more relevant in understanding the meaning of **"bank"**.
- Less attention is given to words that are not directly related (e.g. "The", "after", "rain", ".").
- This helps the model capture the correct context and infer that **"bank"** means the side of a river.

Another Example

Consider the sentence: "I went to the **bank'** to deposit some money."

	I	went	to	the	**bank**	to	**deposit**	some	money	.
	w_1	w_2	w_3	w_4	w_5	w_6	w_7	w_8	w_9	w_{10}
Attention weight (for w_5 'bank')	0.01	0.02	0.05	0.08	0.10	0.05	**0.32**	**0.20**	**0.15**	0.02

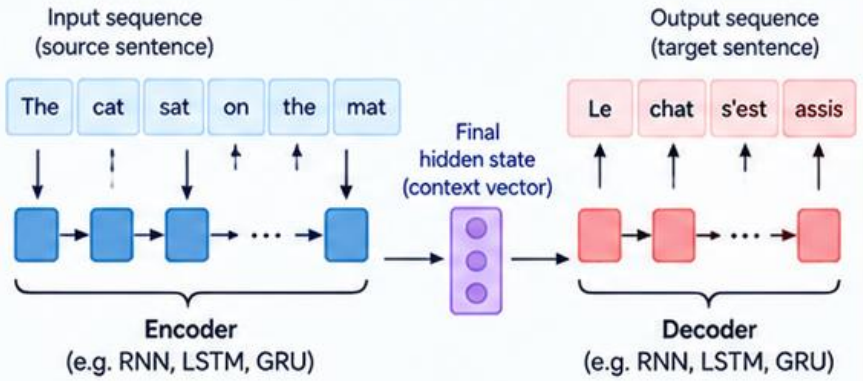
Higher attention on relevant words

Interpretation

- The model focuses more on **"deposit"** and **"money"**.
- This context helps the model understand that **"bank"** refers to a financial institution.
- Attention allows the model to look at different parts of the sentence and give more importance to the relevant words.

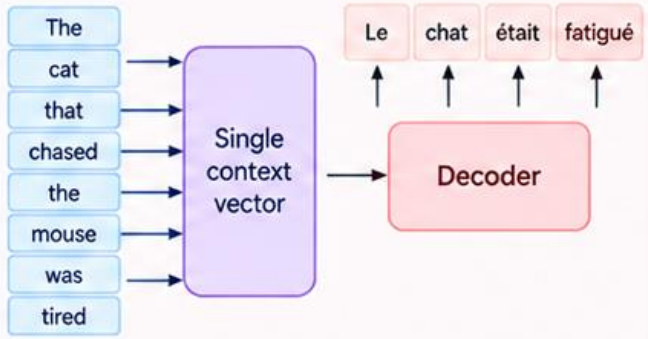
From Encoder–Decoder to Attention

1. Traditional Encoder–Decoder (Without Attention)



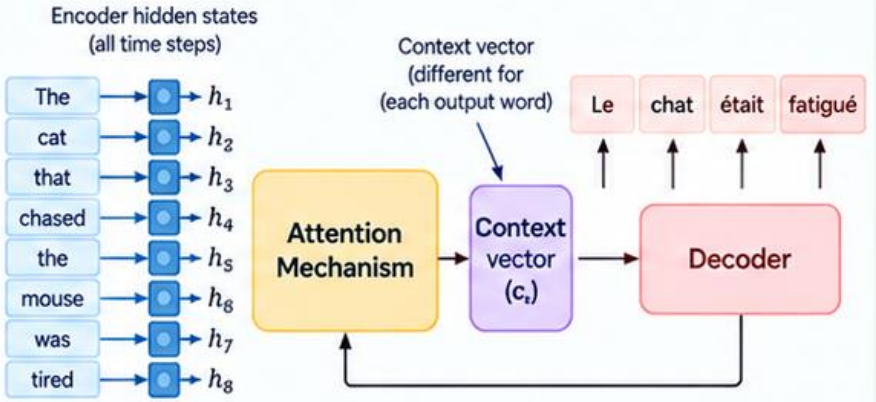
- The encoder processes the input sequence and produces a single fixed-size vector (final hidden state).
- This vector is used to generate the entire output sequence.
- All source information must be compressed into this single vector.

2. Limitations of the Traditional Approach



- All input information is compressed into a single fixed-size vector.
- Important information from earlier words may be lost, especially for long sentences.
- The decoder has no direct access to the encoder hidden states.
- This can lead to poorer translations and less accurate predictions.

3. Introducing Attention



- The decoder can attend to all encoder hidden states, not just the final one.
- Different encoder states receive different importance (attention weights) when generating each output word.
- This allows the model to focus on the most relevant parts of the input sequence and improves performance, especially for long sentences.

Key Takeaway

Attention removes the bottleneck of a single fixed-size context vector by allowing the decoder to directly access all encoder hidden states and focus on the most relevant information when generating each output word.

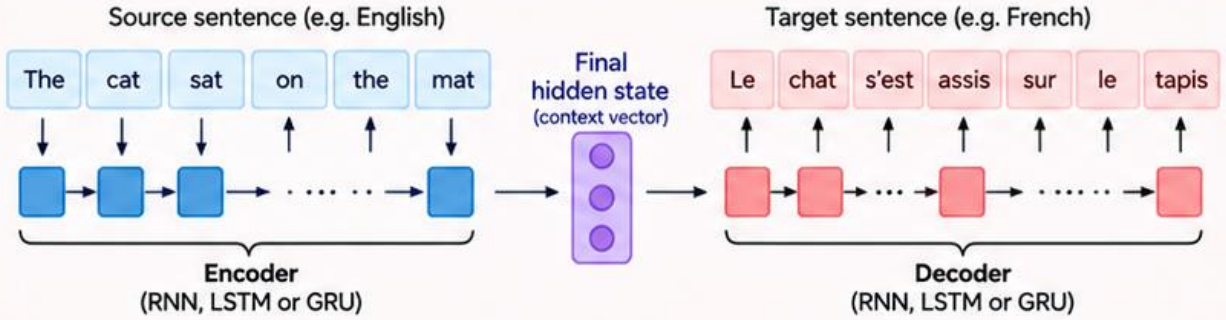
Attention provides a flexible and dynamic way to use the input sequence during decoding, leading to better understanding and more accurate predictions in NLP tasks.

The Encoder–Decoder Bottleneck

In the traditional encoder–decoder model, the entire input sequence is compressed into a single fixed-size context vector. This creates a bottleneck, especially for long sentences, as important information from earlier words may be lost.

Traditional Encoder–Decoder (Without Attention)

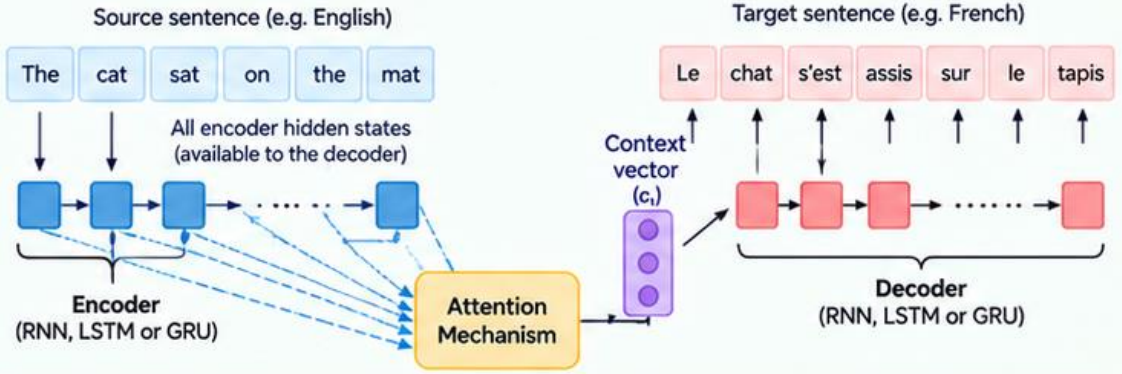
All source information is compressed into a single context vector.



- The encoder processes the input sequence and produces a single fixed-size context vector (the final hidden state).
- This vector must contain all the information from the entire input sentence.
- For long or complex sentences, the fixed-size vector can become a bottleneck, leading to loss of important information.
- The decoder has no direct access to individual source words or earlier hidden states.

With Attention (no bottleneck)

The decoder can access all encoder hidden states and focus on the most relevant ones at each step.



- Instead of relying only on a single fixed-size vector, the decoder can attend to all encoder hidden states.
- It computes attention weights to identify the most relevant source words for the current target word.
- This removes the bottleneck and allows better handling of long sentences and complex dependencies.

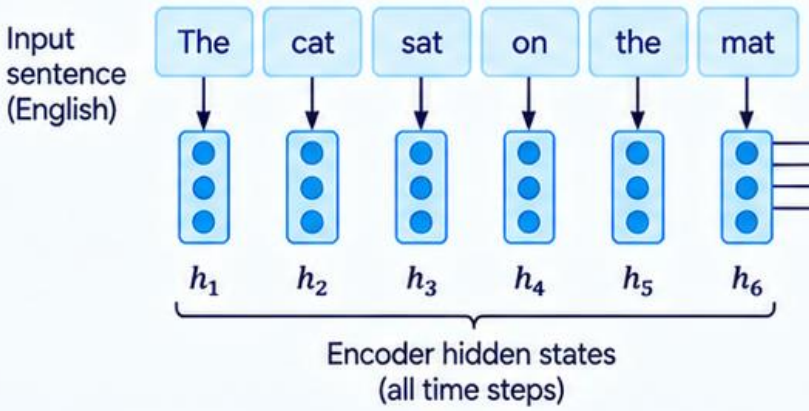
Key Takeaway

The traditional encoder–decoder model compresses the entire input sequence into a single vector, which can lead to a loss of important information. Attention removes this bottleneck by allowing the decoder to access and focus on all relevant encoder hidden states.

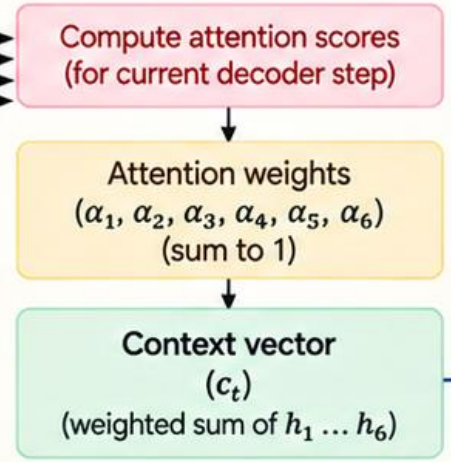
Attention Architecture — Big Picture

Attention allows the decoder to directly access and focus on the most relevant encoder hidden states when generating each output word.
It produces a context vector (a weighted sum of encoder states) for each decoding step, which is used to predict the next word.

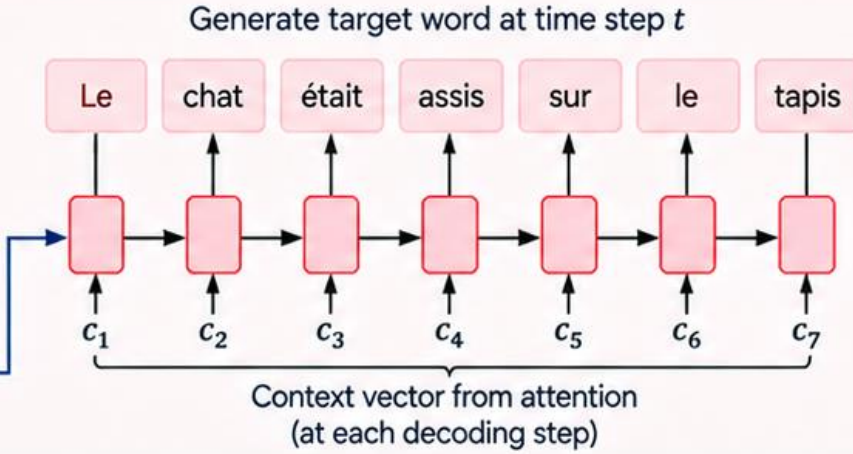
Encoder



Attention Mechanism



Decoder



Example: Attention Weights for Each French Word

	The	cat	sat	on	the	mat
Le	0.70	0.05	0.05	0.05	0.10	0.05
chat	0.05	0.75	0.05	0.05	0.05	0.05
était	0.10	0.10	0.60	0.10	0.05	0.05
assis	0.10	0.10	0.65	0.05	0.05	0.05
sur	0.05	0.05	0.05	0.70	0.10	0.05
le	0.05	0.05	0.05	0.10	0.70	0.05
tapis	0.05	0.05	0.05	0.10	0.10	0.70

Each French word attends more to the relevant English word(s).

Decoding Process (Step by Step)

1. **Le** — Attends mainly to "The" → generates "Le"
2. **chat** — Attends mainly to "cat" → generates "chat"
3. **était** — Attends mainly to "sat" → generates "était"
4. **assis** — Attends mainly to "sat" → generates "assis"
5. **sur** — Attends mainly to "on" → generates "sur"
6. **le** — Attends mainly to "the" → generates "le"
7. **tapis** — Attends mainly to "mat" → generates "tapis"

Final Output (Translation)

Input (English)

The cat sat on the mat.

Output (French)

Le chat était assis sur le tapis.

Key Takeaway

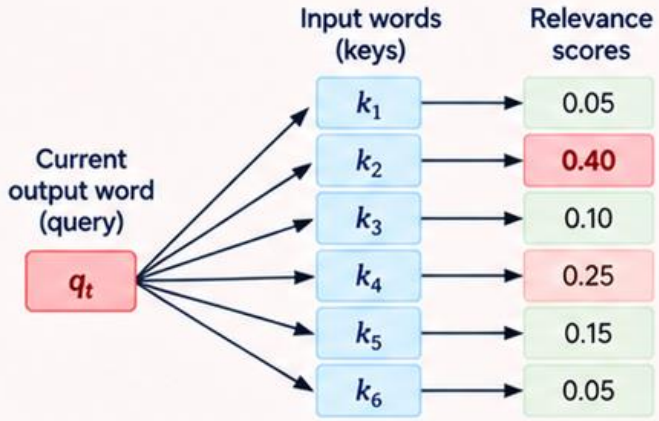
The attention mechanism does not directly produce the final sentence. It produces a context vector for each decoding step, which helps the decoder generate the next word.

What Does an Attention Mechanism Do?

An attention mechanism learns to focus on the most relevant parts of the input sequence when generating each output word.
It computes a weighted sum of all input representations, where the weights (attention scores) indicate how relevant each input word is to the current word.

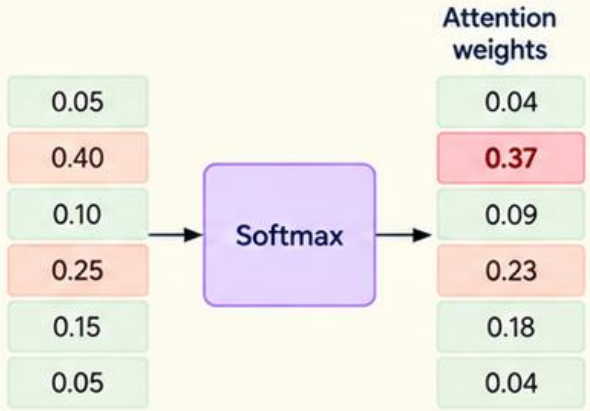
1 Compute Relevance (Attention Scores)

For the current output word, the model compares it with all input words to compute a relevance score (e.g. using dot product, additive or scaled dot-product).



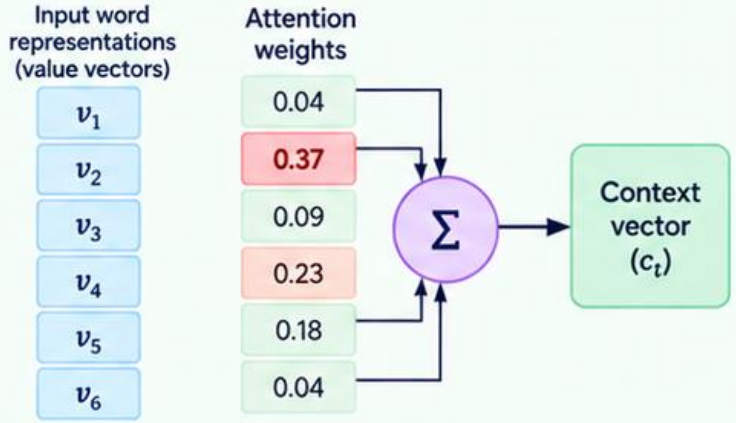
2 Convert to Attention Weights

The relevance scores are normalised using a softmax function to obtain attention weights that sum to 1.



3 Compute Context Vector

The attention weights are used to take a weighted sum of all input representations (value vectors), producing a context vector that captures the most relevant information for the current output word.



Key Takeaways

- Attention learns how much to focus on each input word for the current output word.
- It allows the model to capture long-range dependencies.
- It produces a dynamic context vector instead of relying on a single fixed-size vector.
- The most relevant words receive higher attention weights, while less relevant words receive lower weights.

Benefits in NLP

- Better handling of long sentences.
- Improved understanding of context and meaning.
- Helps disambiguate words (e.g. the meaning of "bank").
- Leads to more accurate and fluent predictions.

SIS 3204Y(5)
Natural Language Processing

Step 1 — Calculate Attention Score

The attention score measures how relevant each input word is to the current output word. It is computed by comparing the decoder's current state (query) with each encoder hidden state (key), typically using a dot product (or scaled dot product). A higher score means higher relevance.

Key Idea

- The decoder's current hidden state (query) represents the word we are trying to generate.
- Each encoder hidden state (key) represents an input word.
- We compare the query with each key to get an attention score that indicates how relevant each input word is.
- We use a dot product (similarity measure).

Vectors for the Example

Decoder query vector for current output word (e.g. generating "était")

q_t = [0.4 0.1 0.3]

Encoder key vectors (one for each input word)

The	cat	sat	on	the	mat
0.2	0.1	0.3	0.0	0.1	0.2
0.1	0.3	0.4	0.2	0.0	0.1
0.0	0.1	0.2	0.3	0.1	0.4

Encoder hidden states (keys)

Dot Product Attention Score

For each input word i, compute the attention score $e_{t,i}$ as the dot product between the query vector q_t and the key vector k_i:

$$e_{t,i} = \boldsymbol{q}_t \cdot \boldsymbol{k}_i = \sum_{j=1}^{d} q_{t,j} \times k_{i,j}$$

- d is the vector dimension (e.g. 3 in this example).
- A higher score means the input word is more relevant to the current output word.
- These scores will be normalised in the next step to obtain attention weights.

Example Calculation (for "était")

Input word	Key vector k_i	Dot product $e_{t,i} = q_t \cdot k_i$
The	[0.2, 0.1, 0.0]	0.4×0.2 + 0.1×0.1 + 0.3×0.0 = 0.08 + 0.01 + 0.00 = **0.09**
cat	[0.1, 0.3, 0.1]	0.4×0.1 + 0.1×0.3 + 0.3×0.1 = 0.04 + 0.03 + 0.03 = **0.10**
sat	[0.3, 0.4, 0.2]	0.4×0.3 + 0.1×0.4 + 0.3×0.2 = 0.12 + 0.04 + 0.06 = **0.22**
on	[0.0, 0.2, 0.3]	0.4×0.0 + 0.1×0.2 + 0.3×0.3 = 0.00 + 0.02 + 0.09 = **0.11**
the	[0.1, 0.0, 0.1]	0.4×0.1 + 0.1×0.0 + 0.3×0.1 = 0.04 + 0.00 + 0.03 = **0.07**
mat	[0.2, 0.1, 0.4]	0.4×0.2 + 0.1×0.1 + 0.3×0.4 = 0.08 + 0.01 + 0.12 = **0.21**

SIS 3204Y(5)
Natural Language Processing

Step 2 — Convert Scores into Attention Weights

The attention scores computed in Step 1 are converted into attention weights using a softmax function. The softmax normalises the scores so that they are positive and sum to 1. These weights indicate how much attention to give to each input word when generating the current output word.

1. Attention Scores (from Step 1)

For the current output word "était", we obtained the following attention scores:

Input word (i)	Score $e_{t,i}$
The	0.09
cat	0.10
sat	0.22
on	0.11
the	0.07
mat	0.21

2. Apply Softmax Function

Convert each score $e_{t,i}$ to an attention weight $\alpha_{t,i}$:

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{n} \exp(e_{t,j})}$$

1. Take the exponential of each score.
2. Divide by the sum of all exponentials.
3. The resulting weights are between 0 and 1 and sum to 1.

3. Calculation Example (for "était")

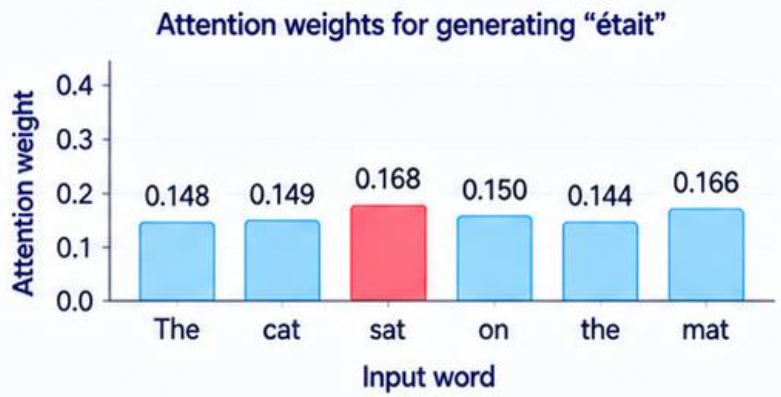
Input word	Score $e_{t,i}$	$\exp(e_{t,i})$	Attention weight $\alpha_{t,i}$
The	0.09	1.094	0.148
cat	0.10	1.105	0.149
sat	0.22	1.246	0.168
on	0.11	1.116	0.150
the	0.07	1.072	0.144
mat	0.21	1.234	0.166
Sum	-	**7.467**	**1.000**

4. Attention Weights (Result)

These are the final attention weights for the input words when generating "était":

Input word	Attention weight $\alpha_{t,i}$
The	0.148
cat	0.149
sat	0.168
on	0.150
the	0.144
mat	0.166

5. Visualisation



Key Takeaways

1. Softmax converts the raw attention scores into a probability distribution.
2. The attention weights show how relevant each input word is to the current output word.
3. The weights are positive and sum to 1.
4. In this example, "sat" receives the highest weight (0.168), so it is the most relevant word for generating "était".

SIS 3204Y(5)
Natural Language Processing

Step 3 — Calculate the Context Vector

The context vector is a weighted sum of all encoder hidden states, where the weights are the attention weights computed in Step 2.
It summarises the most relevant information from the input sentence for the current output word.

1. Encoder Hidden States

Each input word has a hidden state vector h_i

Input word (i)	Hidden state h_i (d = 3)
The	[0.5, 0.1, 0.3]
cat	[0.2, 0.8, 0.1]
sat	[0.4, 0.5, 0.6]
on	[0.1, 0.3, 0.4]
the	[0.3, 0.2, 0.1]
mat	[0.2, 0.1, 0.5]

2. Attention Weights (from Step 2)

For generating "était", the attention weights are:

Input word (i)	Attention weight $\alpha_{t,i}$
The	0.148
cat	0.149
sat	0.168
on	0.150
the	0.144
mat	0.166

3. Compute the Context Vector

The context vector c_t is the weighted sum of all encoder hidden states:

$$c_t = \sum_{i=1}^{n} \alpha_{t,i}\, h_i$$

1. Multiply each hidden state h_i by its attention weight $\alpha_{t,i}$.
2. Sum all the weighted hidden states.
3. The result is the context vector c_t, with the same dimension d (e.g. 3 in this example).

4. Calculation Example (for "était")

Multiply each encoder hidden state by its attention weight, then sum them up.

Input word	$\alpha_{t,i}$	Hidden state h_i	$\alpha_{t,i} \times h_i$ (weighted vector)
The	0.148	[0.5, 0.1, 0.3]	[0.074, 0.015, 0.044]
cat	0.149	[0.2, 0.8, 0.1]	[0.030, 0.119, 0.015]
sat	0.168	[0.4, 0.5, 0.6]	[0.067, 0.084, 0.101]
on	0.150	[0.1, 0.3, 0.4]	[0.015, 0.045, 0.060]
the	0.144	[0.3, 0.2, 0.1]	[0.043, 0.029, 0.014]
mat	0.166	[0.2, 0.1, 0.5]	[0.033, 0.017, 0.083]
Sum = c_t	1.000	-	**[0.262, 0.289, 0.317]**

5. Resulting Context Vector

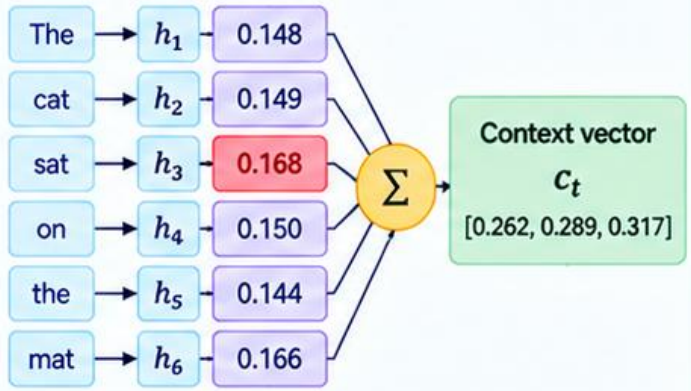
The context vector for generating "était" is:

$$c_t = [0.262, 0.289, 0.317]$$

- It is a single vector that summarises the relevant information from all input words.
- Words with higher attention weights (e.g. "sat") contribute more to this vector.
- This context vector is then used by the decoder (together with the previous output word) to generate "était".

6. Visualisation

The context vector is a weighted sum of all encoder hidden states.



Attention — Complete Worked Example

We show how the attention mechanism computes the context vector for generating the French word **"était"** (when translating "The cat sat on the mat"). The same process is repeated for every output word.

1. Encoder Hidden States

The encoder processes the input sentence and produces a hidden state h_i for each input word.

Position i	Input word	Hidden state h_i (d = 3)
1	The	[0.5, 0.1, 0.3]
2	cat	[0.2, 0.8, 0.1]
3	sat	[0.4, 0.5, 0.6]
4	on	[0.1, 0.3, 0.4]
5	the	[0.3, 0.2, 0.1]
6	mat	[0.2, 0.1, 0.5]

2. Decoder Query Vector

To generate the word "était", the decoder uses its current hidden state (query vector) q_t.

$$q_t = [0.4, 0.1, 0.3]$$

(decoding the word "était")

3. Calculate Attention Scores

Compute a score for each input word using the dot product: $e_{t,i} = q_t \cdot h_i$.

i	Input word	Calculation	Score $e_{t,i}$
1	The	0.4×0.5 + 0.1×0.1 + 0.3×0.3	0.29
2	cat	0.4×0.2 + 0.1×0.8 + 0.3×0.1	0.18
3	sat	0.4×0.4 + 0.1×0.5 + 0.3×0.6	**0.36**
4	on	0.4×0.1 + 0.1×0.3 + 0.3×0.4	0.19
5	the	0.4×0.3 + 0.1×0.2 + 0.3×0.1	0.19
6	mat	0.4×0.2 + 0.1×0.1 + 0.3×0.5	0.23

4. Convert Scores to Attention Weights

Apply the softmax function:

$$\alpha_{t,i} = \frac{e^{e_{t,i}}}{\sum_{j=1}^{6} e^{e_{t,j}}}$$

i	Input word	Score $e_{t,i}$	$e^{e_{t,i}}$	Weight $\alpha_{t,i}$
1	The	0.29	1.336	0.158
2	cat	0.18	1.197	0.142
3	sat	0.36	1.434	**0.170**
4	on	0.19	1.209	0.143
5	the	0.19	1.209	0.143
6	mat	0.23	1.259	0.149
Sum	-	-	**7.644**	**1.000**

5. Calculate the Context Vector

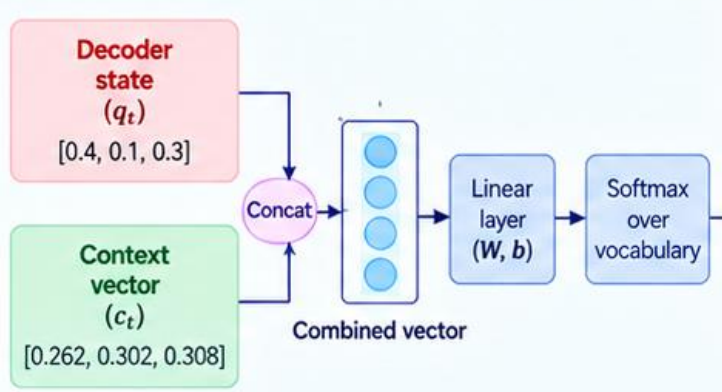
Compute the weighted sum of all encoder hidden states:

$$c_t = \sum_{i=1}^{6} \alpha_{t,i} \, h_i$$

i	Input word	$\alpha_{t,i}$	Hidden state h_i	$\alpha_{t,i} \times h_i$
1	The	0.158	[0.5, 0.1, 0.3]	[0.079, 0.016, 0.047]
2	cat	0.142	[0.2, 0.8, 0.1]	[0.028, 0.114, 0.014]
3	sat	0.170	[0.4, 0.5, 0.6]	[0.068, 0.085, 0.102]
4	on	0.143	[0.1, 0.3, 0.4]	[0.014, 0.043, 0.057]
5	the	0.143	[0.3, 0.2, 0.1]	[0.043, 0.029, 0.014]
6	mat	0.149	[0.2, 0.1, 0.5]	[0.030, 0.015, 0.074]
Sum = c_t		**1.000**	-	**[0.262, 0.302, 0.308]**

6. Use the Context Vector to Generate the Word

The context vector c_t is combined with the decoder state to predict the next word using a linear layer and softmax over the vocabulary.



Output probabilities (illustrative)

Word	Probability
Le	0.05
chat	0.08
était	**0.62**
assis	0.12
sur	0.08
...	...

7. Result

The model predicts **"était"** as the next word because the context vector gives the most relevant information, with the highest attention weight on the input word "sat".

Key Takeaways

- Attention scores measure relevance between the current output word and each input word.
- Softmax converts scores into weights that sum to 1.
- The context vector is a weighted sum of encoder hidden states.
- The context vector helps the decoder predict the next word.

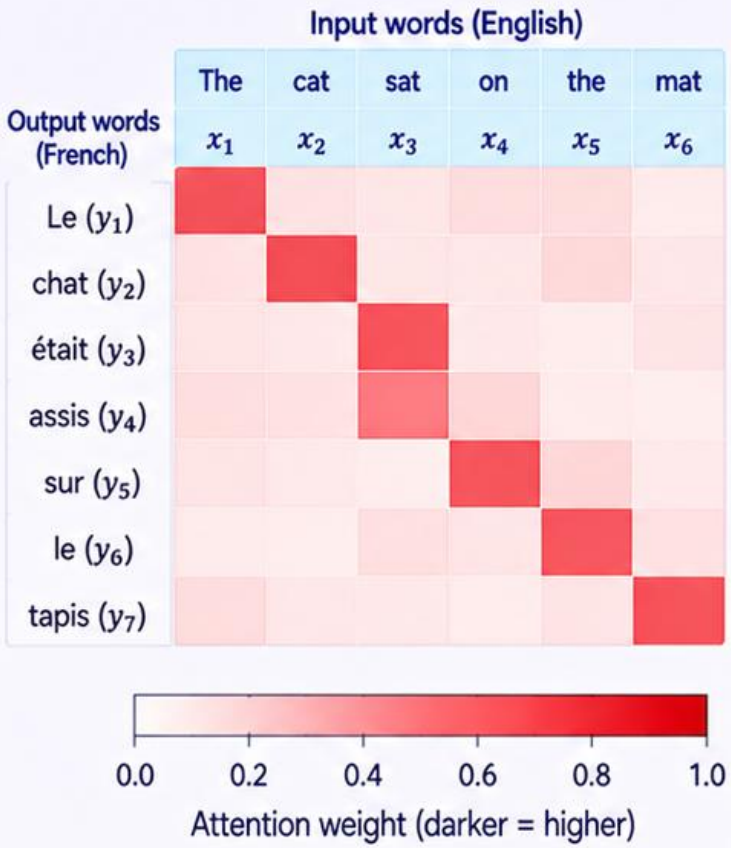
SIS 3204Y(5)
Natural Language Processing

Interpreting Attention Weights

Attention weights tell us which input words the model focuses on when generating each output word. By analysing the attention weights, we can understand how the model aligns the input and output words and which parts of the input sentence are most relevant at each step.

1. Attention Heatmap

A heatmap visualises the attention weights between the output words and the input words.



2. Reading the Attention Pattern

Each row shows the attention weights for one output word. The darker the colour, the more the model attends to that input word.

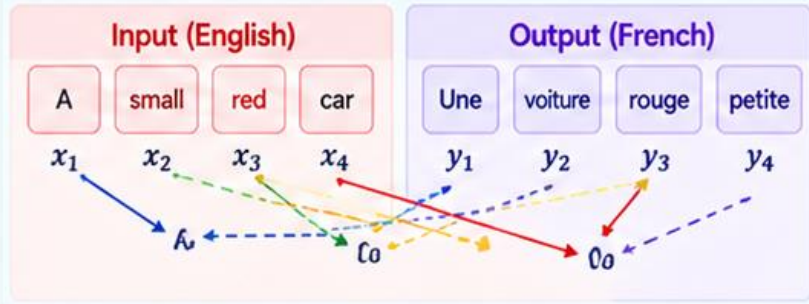
Output word (French)	Main input word(s) attended to	Interpretation
Le	The	Attends mostly to "The" (translates to article "Le").
chat	cat	Attends to "cat" (translates to "chat").
était	sat	Attends to "sat" (translates to "était").
assis	sat	Also attends to "sat" (completes the verbal phrase).
sur	on	Attends to "on" (translates to "sur").
le	the	Attends to "the" (translates to article "le").
tapis	mat	Attends to "mat" (translates to "tapis").

3. What Attention Weights Tell Us

1. They show alignment between input and output words (which words correspond to each other).
2. They help the model focus on the most relevant words for each output word.
3. Some output words may attend to multiple input words (e.g. verb phrases or reordering).
4. **Attention weights are dynamic:** they change for every output word and every input sentence.
5. We can analyse attention to understand model behaviour and diagnose errors (e.g. wrong alignment).

4. Additional Example: Reordering

Attention helps the model handle word order differences between languages. In the English sentence below, the adjective comes before the noun, while in French it comes after the noun.



SIS 3204Y(5)
Natural Language Processing

Attention Changes at Each Decoding Step

During generation, the decoder produces one output word at a time. At each step, it computes a new attention distribution over the input words, so the focus can change depending on the current output word.

Input sentence (English):

The	cat	sat	on	the	mat
x_1	x_2	x_3	x_4	x_5	x_6

→

Le	chat	était	assis	sur	le	tapis
y_1	y_2	y_3	y_4	y_5	y_6	y_7

1. Generate "Le" (y_1)

Le

Input words	Attention weights
The	0.72
cat	0.08
sat	0.05
on	0.04
the	0.07
mat	0.04

Focuses mainly on "The" because "Le" translates to "The".

2. Generate "chat" (y_2)

chat

Input words	Attention weights
The	0.06
cat	0.78
sat	0.07
on	0.03
the	0.04
mat	0.02

Focuses mainly on "cat" because "chat" translates to "cat".

3. Generate "était" (y_3)

était

Input words	Attention weights
The	0.05
cat	0.10
sat	0.68
on	0.06
the	0.06
mat	0.05

Focuses mainly on "sat" because "était" translates to "was".

4. Generate "assis" (y_4)

assis

Input words	Attention weights
The	0.04
cat	0.12
sat	0.71
on	0.06
the	0.05
mat	0.02

Also focuses on "sat" because "assis" means "sitting".

5. Generate "sur" (y_5)

sur

Input words	Attention weights
The	0.03
cat	0.05
sat	0.09
on	0.74
the	0.06
mat	0.03

Focuses mainly on "on" because "sur" translates to "on".

6. Generate "le" (y_6)

le

Input words	Attention weights
The	0.63
cat	0.06
sat	0.06
on	0.06
the	0.14
mat	0.05

Focuses mainly on the second "the" because "le" translates to "the".

7. Generate "tapis" (y_7)

tapis

Input words	Attention weights
The	0.05
cat	0.04
sat	0.05
on	0.06
the	0.07
mat	0.73

Focuses mainly on "mat" because "tapis" translates to "mat".

Key Takeaways

1. The attention distribution is recomputed at every decoding step.
2. The model focuses on the most relevant input word(s) for the current output word.
3. Different output words attend to different input words.
4. Attention helps the model handle word order differences between languages (e.g. adjective before noun in English, after noun in French).

Attention in Machine Translation

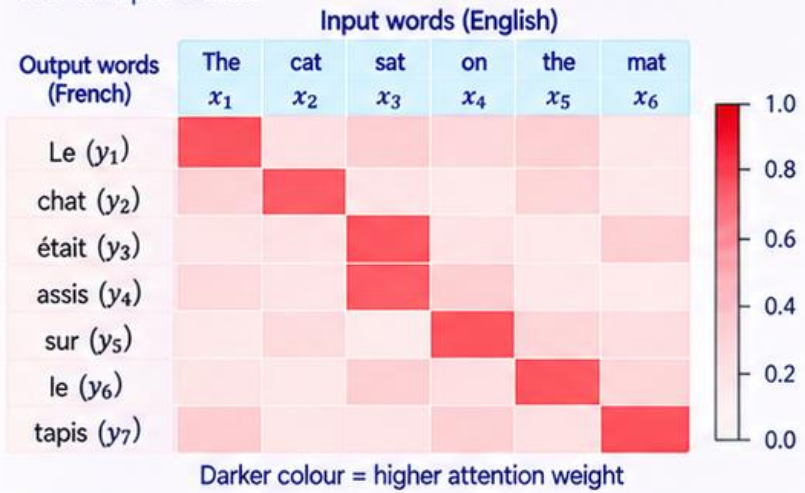
Attention allows the model to focus on the most relevant input words when translating a sentence. It learns soft alignments between the source (input) and target (output) words, which helps generate more accurate and fluent translations.

1. Example Sentence

Input (English):	The	cat	sat	on	the	mat
	x_1	x_2	x_3	x_4	x_5	x_6
Output (French):	Le	chat	était	assis	sur	le	tapis
	y_1	y_2	y_3	y_4	y_5	y_6	y_7

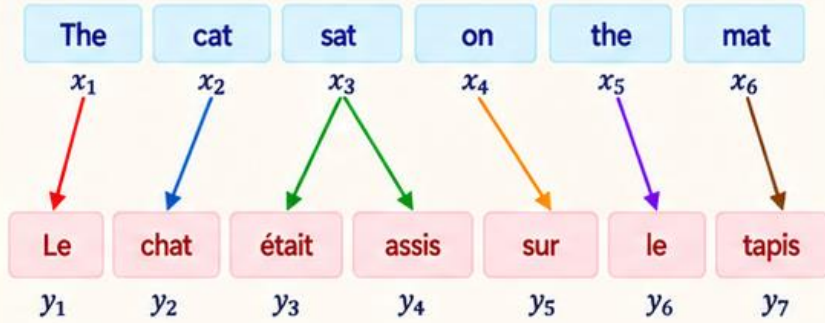
2. Attention Heatmap

Each row shows the attention weights for one output word over all input words.



3. Visualising the Alignments

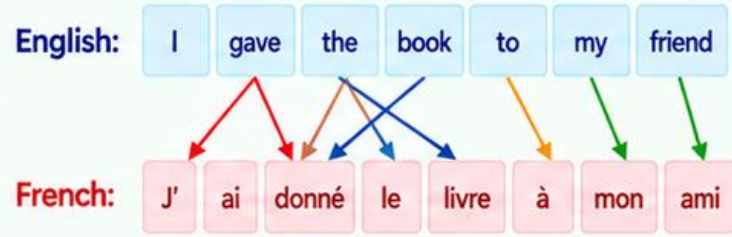
Attention shows which input word(s) each output word relies on.



Output word (French)	Main attended input word(s) (English)	Interpretation
Le	The	"Le" corresponds to "The".
chat	cat	"chat" corresponds to "cat".
était	sat	"était" (was) corresponds to "sat".
assis	sat	"assis" (sitting) also uses "sat".
sur	on	"sur" corresponds to "on".
le	the	"le" corresponds to the second "the".
tapis	mat	"tapis" corresponds to "mat".

4. Handling Reordering

Attention helps the model handle different word orders between languages.



5. Handling Multi-Word Translations

Some words in the target language may correspond to multiple input words or vice versa.



Even though word order is similar here, attention allows the model to learn the correct alignments automatically.

Key Takeaways

1. Attention learns soft alignments between input and output words.
2. It helps the model focus on the most relevant input words when generating each output word.
3. It can handle reordering and multi-word translations.
4. Attention improves translation accuracy and fluency by using the right context at each step.

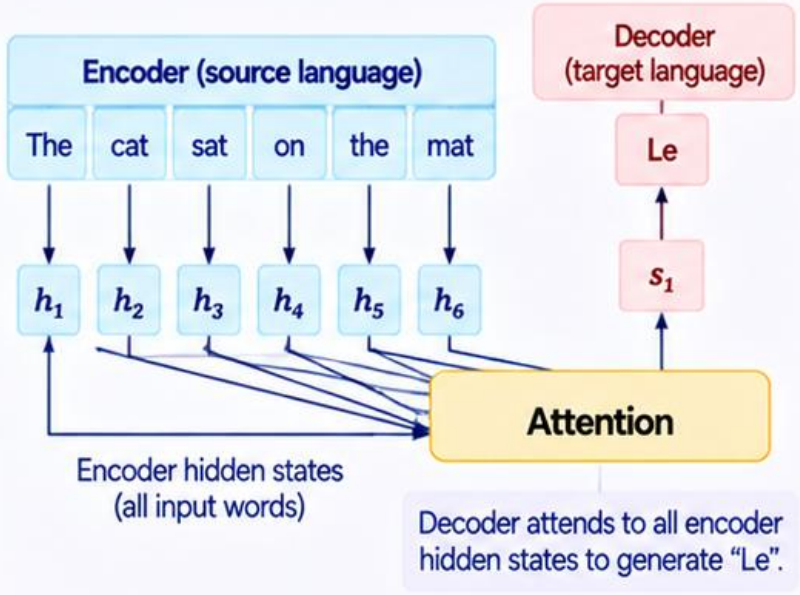
SIS 3204Y(5)
Natural Language Processing

From Attention to Self-Attention

The attention mechanism we have seen in encoder–decoder models aligns two different sequences (e.g. source and target in machine translation). Self-attention generalises this idea by allowing each word in the same sequence to attend to all other words in that sequence.

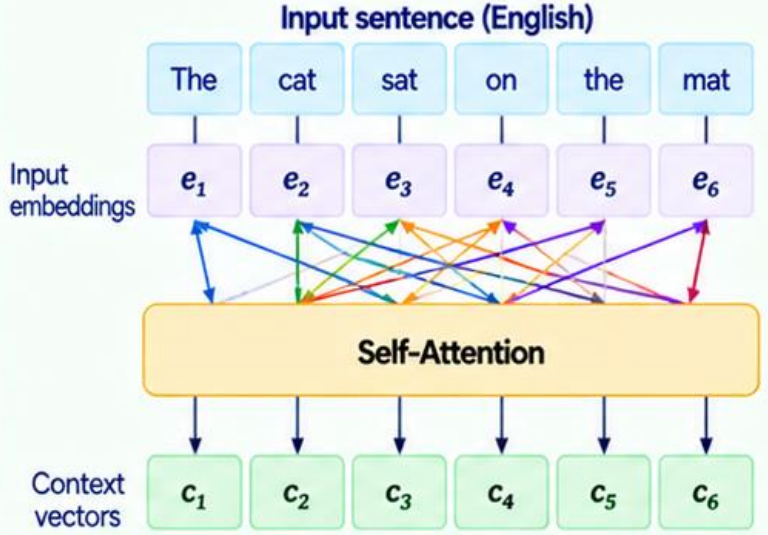
1. Cross-Attention in Encoder–Decoder

In machine translation, the decoder attends to the encoder hidden states to focus on relevant source (input) words when generating each target (output) word.



2. Self-Attention in a Single Sequence

In transformer models, each word attends to all other words in the same sequence to capture the context from the entire sentence.



Each word attends to all other words in the same sentence to produce a context-aware representation.

3. Difference in What They Attend To

Cross-attention and self-attention differ in the source of the information they attend to.

Cross-Attention (enc–dec)

- Attends to a different sequence.
- Used in encoder–decoder models (e.g. machine translation).
- Decoder attends to encoder hidden states (source sentence).
- Helps align and generate output words based on relevant input words.

Self-Attention (Transformer)

- Attends to the same sequence.
- Used in transformer models (e.g. BERT, GPT).
- Each word attends to all other words in the same sentence.
- Captures both local and long-range dependencies.
- Produces context-aware representations for every word in the sequence.

Key Takeaways

1. Cross-attention aligns two different sequences (e.g. source and target in machine translation).
2. Self-attention allows each word to attend to all other words in the same sequence.
3. Self-attention enables the model to capture long-range dependencies effectively.
4. Self-attention is the core mechanism used in transformer models (e.g. BERT, GPT).

SIS 3204Y(5)
Natural Language Processing

What is Self-Attention?

Self-attention allows each word in a sequence to attend to **all other words in the same sequence.** It computes how much each word should focus on the other words, so that each word can build a context-aware representation that captures the meaning of the whole sentence.

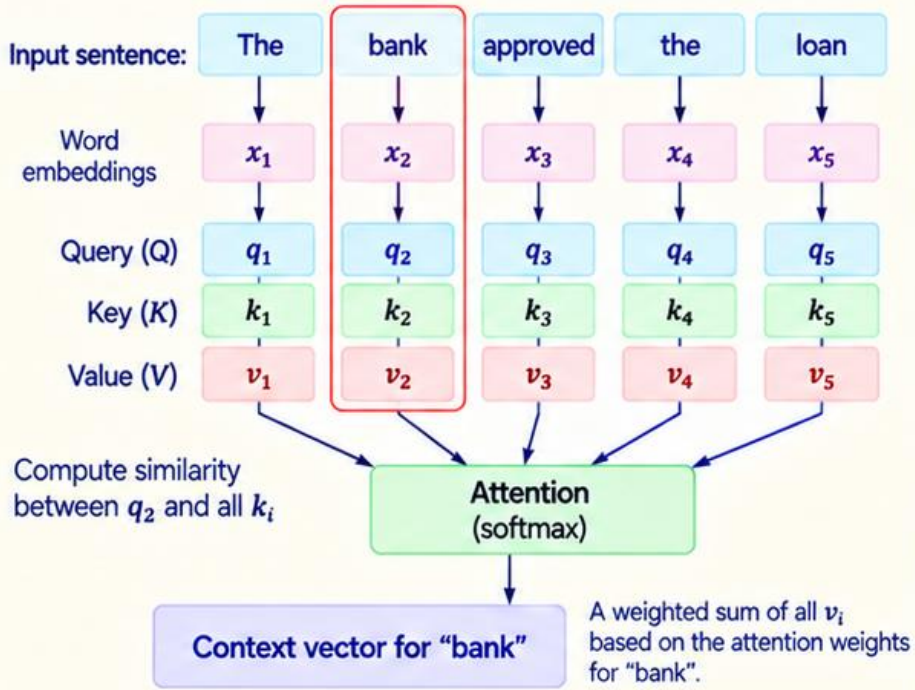
1. Core Idea

For each word, self-attention measures the similarity between that word and every other word in the same sentence. Words that are more related receive higher attention weights.

Self-attention =
attention within the same sequence
(instead of between two different sequences).

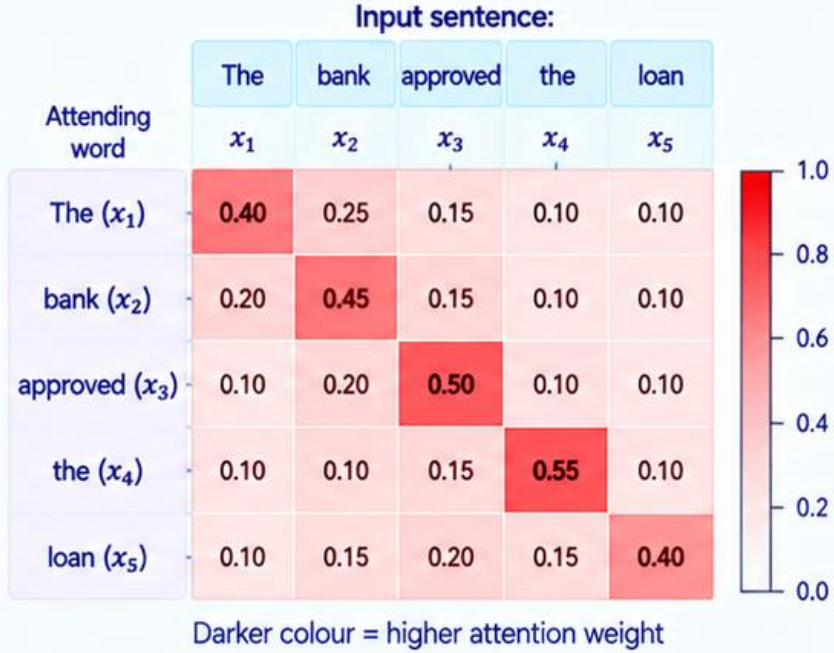
2. Self-Attention for One Word

Example: Consider the word "bank" in the sentence:
"The bank approved the loan".



3. Attention Weights for Each Word

Each row shows which words each word attends to.
(Example values only)



Darker colour = higher attention weight

Key Takeaways

1. Self-attention lets each word attend to all other words in the same sequence.
2. It computes similarity between a query and all keys, converts them to attention weights and takes a weighted sum of the values.
3. Each word gets a context-aware representation that captures information from the entire sentence.
4. Self-attention helps the model understand long-range dependencies and word relationships more effectively.

SIS 3204Y(5)
Natural Language Processing

Self-Attention — NLP Example

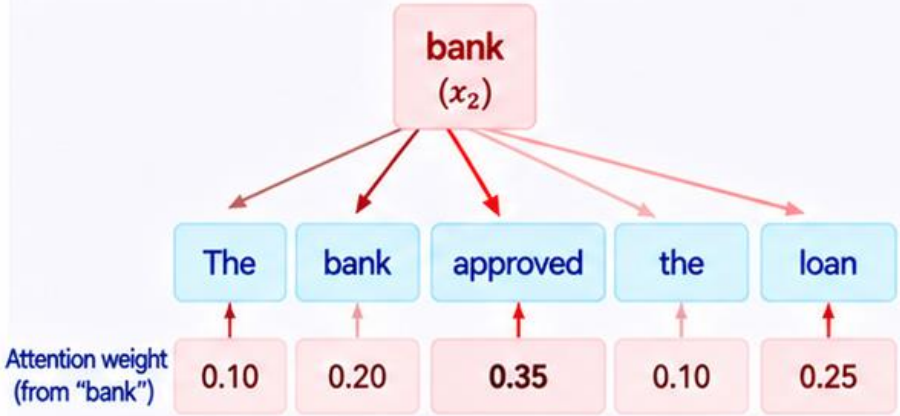
Self-attention allows each word in a sentence to look at **all other words in the** same sentence and focus on the most relevant ones.
This helps the model understand the meaning of each word based on its context.

1. Example Sentence

The	bank	approved	the	loan
x_1	x_2	x_3	x_4	x_5

2. Focus on One Word: "bank"

The model computes how much the word "bank" attends to each word in the same sentence.



3. Attention Weights for "bank"

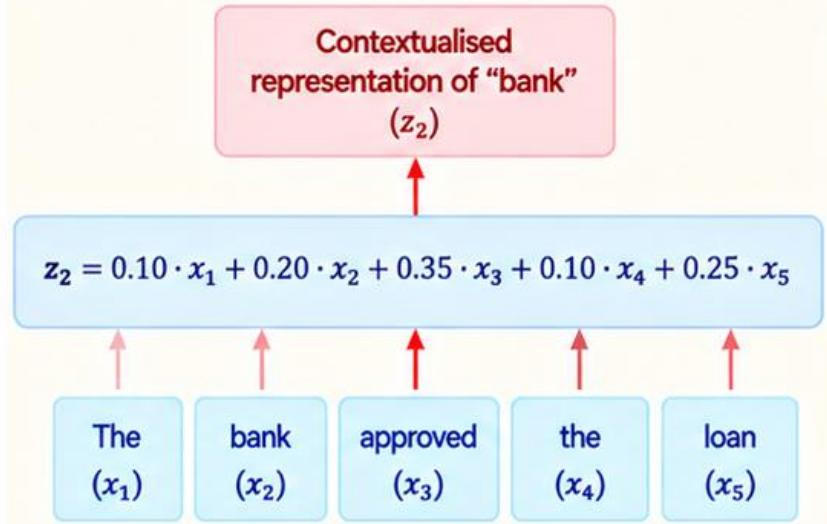
The attention weights show which words provide useful context for understanding "bank".

Word	Attention weight (from "bank")
The (x_1)	0.10
bank (x_2)	0.20
approved (x_3)	0.35
the (x_4)	0.10
loan (x_5)	0.25

Interpretation: The word "bank" pays more attention to "approved" and "loan" because they help indicate that "bank" refers to a financial institution (related to loans), not a river bank.

4. Result: Contextualised Representation

Using these attention weights, the model creates a new representation for "bank" as a weighted combination of the other words' information.



The resulting representation of "bank" captures the meaning of the word based on its context in the whole sentence.

Key Takeaways

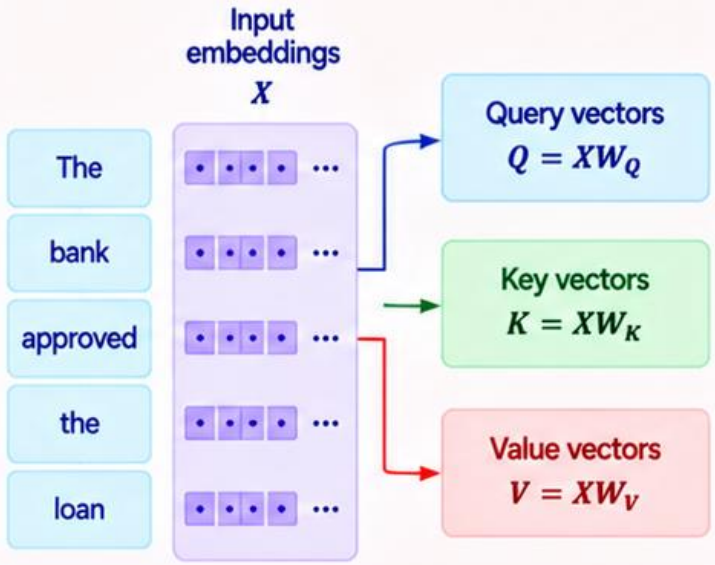
1. In self-attention, each word attends to all other words in the same sentence.
2. The attention weights indicate which words are most relevant for understanding a word's meaning.
3. The final representation (contextualised embedding) combines information from all words based on these weights.
4. This helps the model understand words that have different meanings depending on their context.

Query, Key and Value — The Main Idea

In self-attention, each word is represented by three different vectors: a **Query (Q)**, a **Key (K)** and a **Value (V)**. These vectors are learned from the input embeddings and are used to compute attention weights and the final output (contextualised representation).

1. From Input to Q, K, V

Each input word is first converted to an embedding vector. This embedding is then transformed into three different vectors: Query, Key and Value using learned linear transformations.



2. What Do Q, K and V Do?

Query (Q)

- Represents the word that is looking for information.
- Used to compare with the keys of all other words to find relevant words.

Key (K)

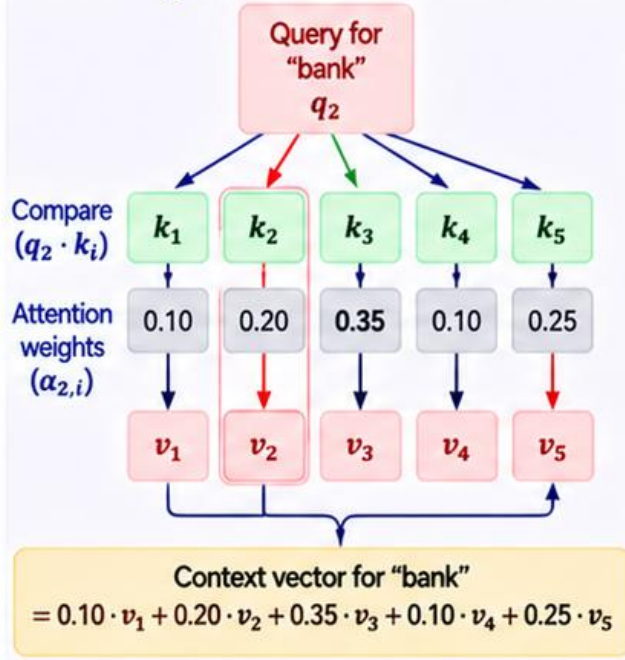
- Represents each word as a potential source of information.
- Used to compute how relevant each word is to the query (by measuring similarity).

Value (V)

- Contains the actual information that will be combined.
- Once the attention weights are computed, the values of relevant words are aggregated to produce the final representation.

3. How They Work Together

For a given word (e.g. "bank"), the query vector is compared with the key vectors of all words to compute attention weights. These weights are then used to take a weighted sum of the value vectors.



4. Intuition

Analogy: asking for help

Each word sends a "query" to ask: "Which other words are useful for understanding me?"

- Keys help decide which words are relevant.
- Values provide the actual information from those words.

Example: "bank"

The word "bank" attends more to "approved" and "loan" because they provide useful context, helping the model understand that "bank" refers to a financial institution.

Key Takeaways

1. Each word has a Query, Key and Value vector.
2. The query compares with all keys to compute attention weights.
3. The values of relevant words are combined using these weights.
4. Q, K and V allow the model to learn which words to focus on and build context-aware representations.

Creating Q, K and V + Calculating Relevance

Each input word is first converted into an embedding vector. These embeddings are then linearly transformed into three different vectors: **Query (Q)**, **Key (K)** and **Value (V)**. The similarity between a query and all keys is computed using a dot product to determine how relevant each word is to the current word.

1. Input Embeddings

Each word in the sentence is represented as an embedding vector (dimension d_{model}).

Sentence:	Input embeddings X $(5 \times d_{model})$			
The	0.2	0.5	−0.1	...
bank	0.1	0.8	0.3	...
approved	−0.4	0.1	0.6	...
the	0.3	−0.2	0.2	...
loan	0.0	0.4	0.5	...

2. Create Query, Key and Value Vectors

The input embeddings are multiplied by three different learned weight matrices to produce **Q**, **K** and V.

Query (Q)

$$Q = XW_Q$$

$X : (5 \times d_{model})$
$W_Q : (d_{model} \times d_k)$
$Q : (5 \times d_k)$

Key (K)

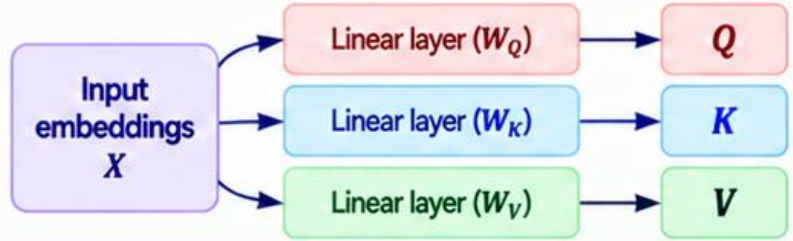
$$K = XW_K$$

$X : (5 \times d_{model})$
$W_K : (d_{model} \times d_k)$
$K : (5 \times d_k)$

Value (V)

$$V = XW_V$$

$X : (5 \times d_{model})$
$W_V : (d_{model} \times d_v)$
$V : (5 \times d_v)$



3. Calculate Relevance (Attention Scores)

For each word, the query vector is compared with the key vectors of all words using a dot product.

$$\text{Attention scores} = QK^T$$
$$(5 \times d_k)\ (5 \times d_k)^T \rightarrow (5 \times 5)$$

Key words (K)

Qey word (Q)	The	bank	approved	the	loan
The	0.8	0.2	-0.1	0.3	0.0
bank	0.1	1.2	0.8	0.1	0.9
approved	0.0	0.7	1.1	0.2	0.8
the	0.3	0.1	0.2	0.9	0.1
loan	0.0	0.6	0.9	0.2	1.0

High relevance

Low relevance

4. What the Scores Mean

- A higher score indicates that the query word finds the key word more relevant.
- For example, the query for “bank” has higher scores with “approved” and “loan” because they provide useful context.
- These scores are then scaled and normalised using softmax to obtain attention weights, which are used to combine the value vectors.

Intuition

Each word asks: “Which other words in the sentence are useful for understanding me?”
The dot product between the query and the keys measures the similarity and helps identify the most relevant words.

SIS 3204Y(5)
Natural Language Processing

Scaled Dot-Product Attention

Scaled dot-product attention computes a **weighted combination of value vectors**, where the weights are based on the similarity between a query and all keys. It allows each word to attend to the most relevant words in the sequence.

Scaled Dot-Product Attention

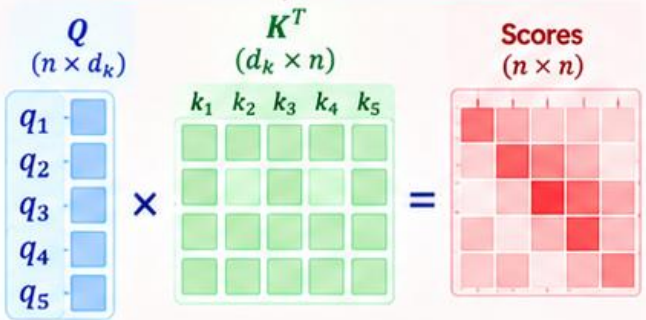
$$Attention(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q : Query matrix $(n \times d_k)$
K : Key matrix $(n \times d_k)$
V : Value matrix $(n \times d_v)$
d_k : Dimension of key/query vectors

1. Compute Similarity Scores

Take the dot product between the query matrix and the transpose of the key matrix.

$$Scores = QK^T \quad (n \times n)$$



Each element s_{ij} measures how similar word i (query) is to word j (key).

2. Scale the Scores

Divide the scores by $\sqrt{d_k}$ to keep the values in a reasonable range.

$$\text{Scaled scores} = \frac{QK^T}{\sqrt{d_k}}$$

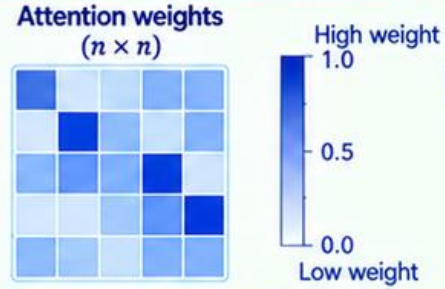


Scaling prevents the dot products from becoming too large, which helps the softmax function work more effectively.

3. Apply Softmax

Convert the scaled scores into attention weights. Each row sums to 1.

$$\text{Attention weights} = \text{softmax}(\text{scaled scores})$$

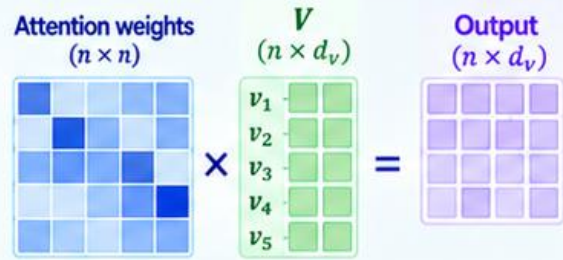


Each row shows how much the current word attends to all words in the sequence.

4. Compute the Output

Multiply the attention weights with the value matrix to get the final output (contextualised representations).

$$\text{Output} = \text{Attention weights} \times V \quad (n \times d_v)$$



Each output vector is a weighted sum of the value vectors, containing information from all relevant words.

Key Takeaways

1. Compute similarity scores using QK^T.
2. Scale by $\sqrt{d_k}$ to keep values in a suitable range.
3. Use softmax to obtain attention weights.
4. Multiply the weights with V to get the final output.

Why Do We Scale by $\sqrt{d_k}$?

- Dot products of high-dimensional vectors can become large.
- Large values make the softmax very peaky (almost one-hot), leading to small gradients and slower learning.
  Scaling keeps the values in a more manageable range.

SIS 3204Y(5)
Natural Language Processing

Self-Attention — Step by Step

Self-attention takes the input embeddings of a sequence and computes a new representation for each word by looking at all other words in the same sequence. The process can be broken down into the following steps:

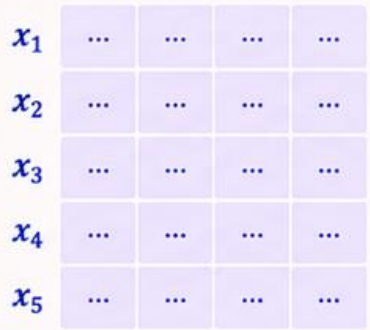
1. Input Embeddings

Each word is represented as an embedding vector of dimension d_{model}.

Input sentence:

The	bank	approved	the	loan
x_1	x_2	x_3	x_4	x_5

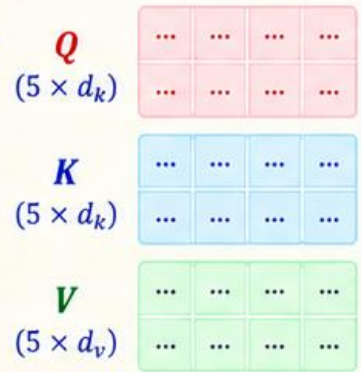
Input embeddings X
$(5 \times d_{model})$



2. Create Q, K and V

The input embeddings are multiplied by three learned weight matrices to produce Query (Q), Key (K) and Value (V) vectors.

$$Q = XW_Q$$
$$K = XW_K$$
$$V = XW_V$$

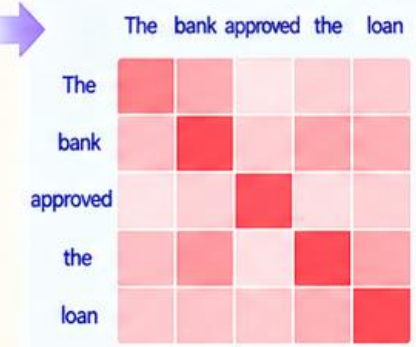


3. Compute Similarity Scores

For each word, compute the dot product between its query and the keys of all words.

$$\text{Scores} = QK^T$$
(5×5)

Similarity scores (QK^T)

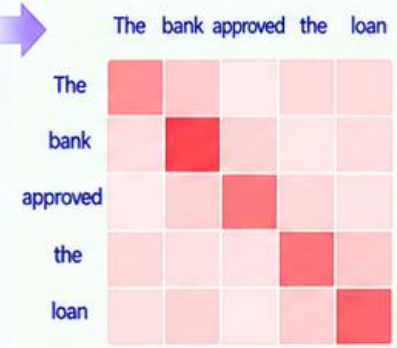


4. Scale the Scores

Divide the scores by $\sqrt{d_k}$ to keep the values in a reasonable range.

$$\text{Scaled scores} = \frac{QK^T}{\sqrt{d_k}}$$
(5×5)

Scaled scores

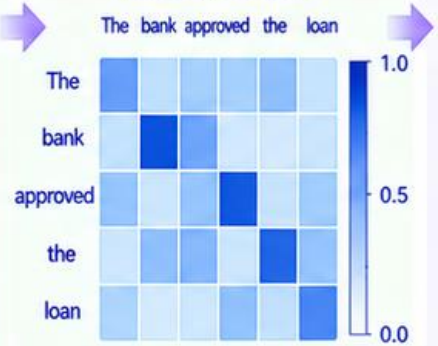


5. Apply Softmax

Apply the softmax function to each row to obtain attention weights that sum to 1.

$$\text{Attention weights} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$
(5×5)

Attention weights

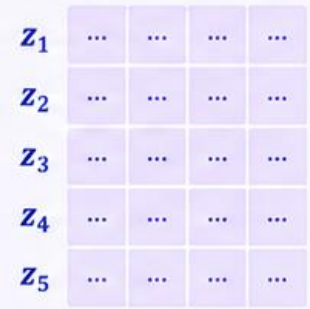


6. Compute Output

Multiply the attention weights with the value vectors to obtain the new representation for each word (context vector).

$$Z = \text{Attention weights} \times V$$
$(5 \times d_v)$

Output (context vectors) Z



Key Takeaways

1. Each word is represented by Query, Key and Value vectors computed from its embedding.
2. The query of each word is compared with the keys of all words to compute similarity scores.
3. The scores are scaled and converted to attention weights using softmax (each row sums to 1).
4. The output for each word is a weighted sum of the value vectors, producing a context-aware representation.

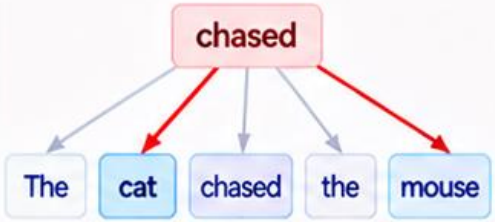
What Can Self-Attention Learn?

Self-attention learns to capture different types of relationships between words by assigning higher weights to relevant words in the sentence.
It can learn local and long-range dependencies, syntactic relationships, and semantic relationships, which help build a rich contextual representation for each word.

1. Syntactic Relationships

Self-attention can learn which words are syntactically related (e.g. subject–verb, verb–object, modifiers).

The **cat** **chased** the mouse .



Attention weights for "chased"

The	cat	chased	the	mouse
0.05	0.35	0.10	0.10	0.40

"chased" attends more to "cat" (subject) and "mouse" (object).

2. Long-Range Dependencies

Self-attention can connect words that are far apart in the sentence, which is difficult for traditional RNNs.

The book that I **bought** yesterday was interesting .



Attention weights for "bought"

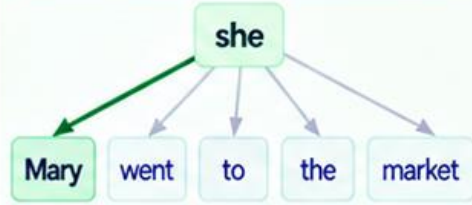
book	that	I	bought	yesterday
0.32	0.10	0.08	0.20	0.30

"bought" attends to "book" and "yesterday", even though they are several words apart.

3. Coreference Resolution

Self-attention can help identify that two words refer to the same entity (e.g. a pronoun and its antecedent).

Mary went to the market because **she** needed milk .



Attention weights for "she"

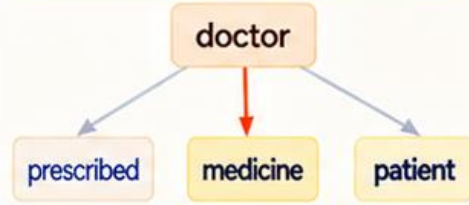
Mary	went	to	the	market
0.60	0.05	0.10	0.15	0.10

"she" attends strongly to "Mary", helping the model understand that they refer to the same person.

4. Semantic Similarity

Self-attention can capture semantic relationships between words with similar meanings or related concepts.

The **doctor** prescribed medicine for the patient .



Attention weights for "doctor"

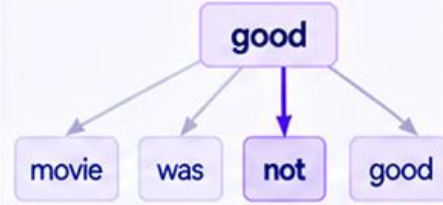
prescribed	medicine	patient
0.20	0.50	0.30

"doctor" attends to "medicine" and "patient" because they are semantically related.

5. Negation and Context

Self-attention helps the model focus on important context words (e.g. negation, contrast) to correctly understand meaning.

The movie was **not** good .



Attention weights for "good"

movie	was	not	good
0.10	0.15	0.55	0.20

"good" attends strongly to "not", which changes the meaning of the sentence.

Key Takeaways

1. Self-attention can learn many different types of relationships between words.
2. It can capture both local and long-range dependencies.
3. The learned attention weights help the model build rich contextual representations.

Intuition

Each word looks at all other words and learns which ones are useful for understanding its meaning. The attention weights are not fixed — they are learned from data during training.

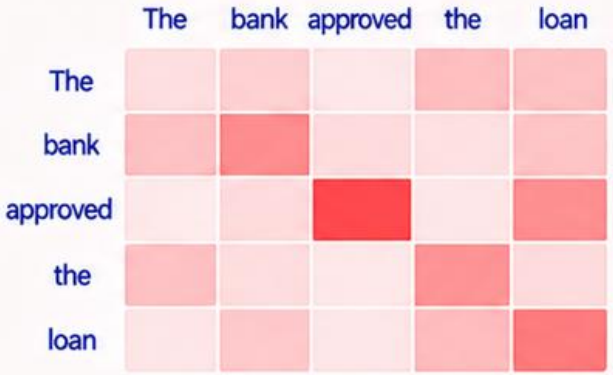
From Single-Head to Multi-Head Attention

A single attention head can only capture one type of relationship at a time. Different types of relationships are important in a sentence (e.g. syntactic, semantic, coreference). Multi-head attention uses **multiple attention heads in parallel**, so that each head can focus on a different aspect of the sentence, and together they provide a richer representation.

1. Single-Head Attention

A single attention head computes one set of attention weights, capturing one dominant type of relationship in tihe sentence.

Attention weights (single head)



This head may focus mainly on syntactic relationships, for example linking “approved” with “bank” and “loan”.

2. Limitation of a Single Head

- It produces only one set of attention weights.
- It may miss other useful relationships (e.g. coreference, semantic similarity).
- A single head cannot focus on multiple aspects at the same time.

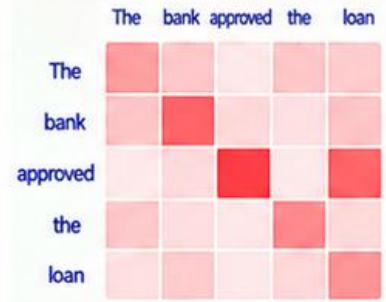
Example:

In the sentence “The bank approved the loan”, a single head may capture that “approved” is related to “bank” and “loan” (syntactic relation), but it may not also capture that “bank” refers to a financial institution (semantic relation) at the same time.

3. Multiple Heads Capture Different Aspects

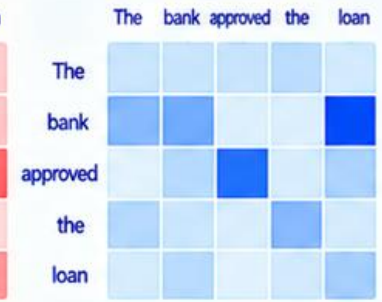
Multi-head attention uses several attention heads in parallel. Each head learns a different set of attention weights, allowing the model to capture different types of relationships in the same layer.

Head 1
(Syntactic relations)



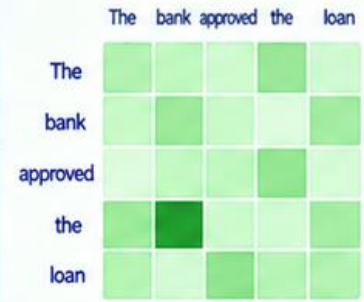
Focuses on syntactic dependencies.
“approved” attends to “bank” and “loan”.

Head 2
(Semantic similarity)



Captures semantic relationships.
“bank” attends to “loan” because they are related in meaning.

Head 3
(Coreference / context)



Helps with coreference and context.
“the” attends to “bank” to identify which entity the pronoun refers to.

Key Takeaways

1. A single attention head can capture only one type of relationship at a time.
2. Multi-head attention uses several heads in parallel, each focusing on a different aspect of the sentence.
3. The outputs from all heads are combined to give a richer contextual representation for each word.

What is Multi-Head Attention?

In practice, we use h attention heads (e.g. $h = 8$) in parallel.

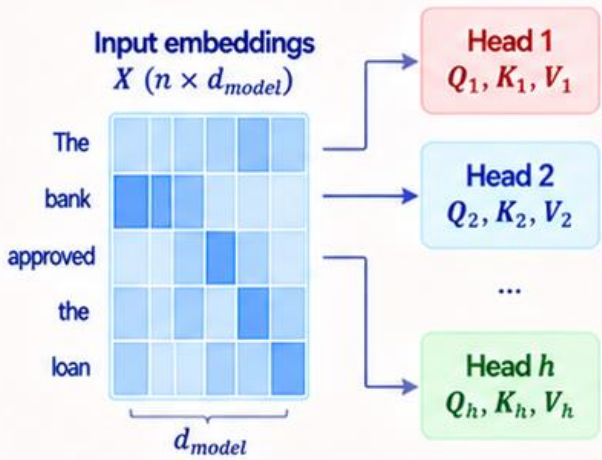
Multi-head attention performs **several attention operations (heads) in parallel.** Each head has its own learned projection matrices, allowing it to focus on different aspects of the sequence. The outputs from all heads are then combined to produce the final representation.

1. Input and Linear Projections

The input embeddings X are projected into multiple sets of Query, Key and Value matrices – one set for each head.

$$Q_i = XW_Q^{(i)} \quad K_i = XW_K^{(i)} \quad V_i = XW_V^{(i)}$$

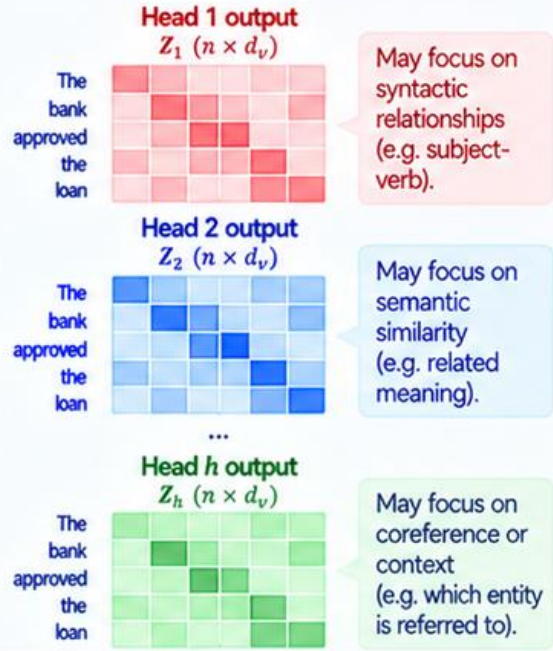
for $i = 1, 2, \ldots, h$ (heads)



2. Attention in Each Head

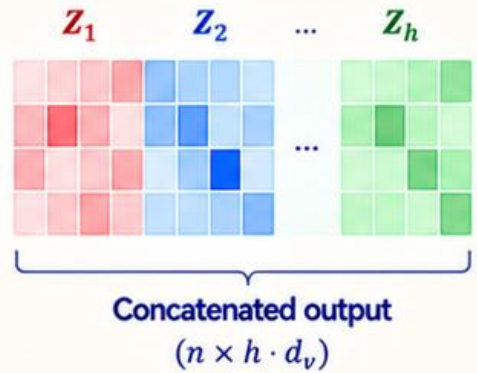
Each head independently computes scaled dot-product attention using its own Q_i, K_i and V_i.

$$\text{Attention}_i = \text{softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i$$



3. Concatenate the Heads

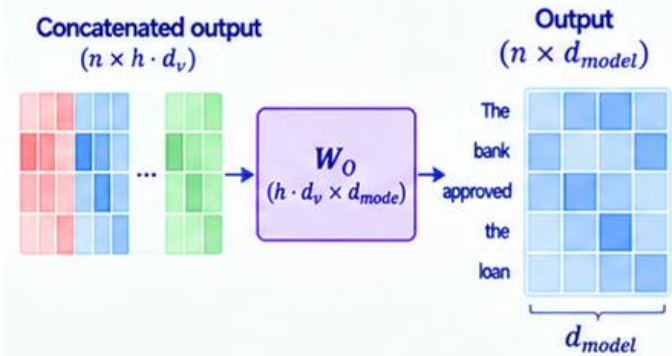
The outputs from all heads are concatenated (side by side) to form a single matrix.



4. Final Linear Projection

The concatenated output is passed through a final linear layer to mix the information from all heads and produce the output.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(Z_1, \ldots, Z_h)\, W_O$$



Key Takeaways

1. Multi-head attention runs several attention heads in parallel, each with its own Q, K, V.
2. Each head can learn different types of relationships (syntactic, semantic, coreference, etc.).
3. The outputs of all heads are concatenated and projected to produce the final representation.

Why Multiple Heads?

- Allows the model to capture different aspects of a sentence.
- Improves the richness and quality of the learned representations.
- More expressive than a single attention head.
- Empirically leads to better performance in language tasks.

From Self-Attention to Transformers

Self-attention is the core building block of the Transformer. It allows the model to look at all words in the sequence and build contextual representations in parallel. A Transformer combines self-attention with additional components such as positional encoding, feed-forward networks, residual connections and layer normalization to create a deep and powerful language model.

1. Self-Attention (Recap)

Self-attention takes the input embeddings and computes a new representation for each word by attending to all other words in the sequence.

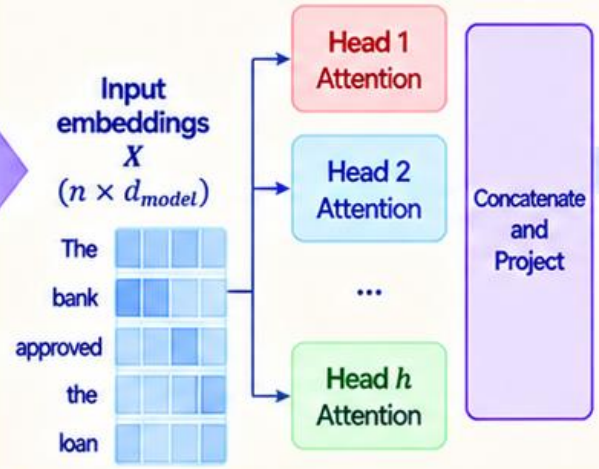
$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^T}{\sqrt{d_k}}\right)\boldsymbol{V}$$



Output: context-aware representations for each word.

2. Multi-Head Attention

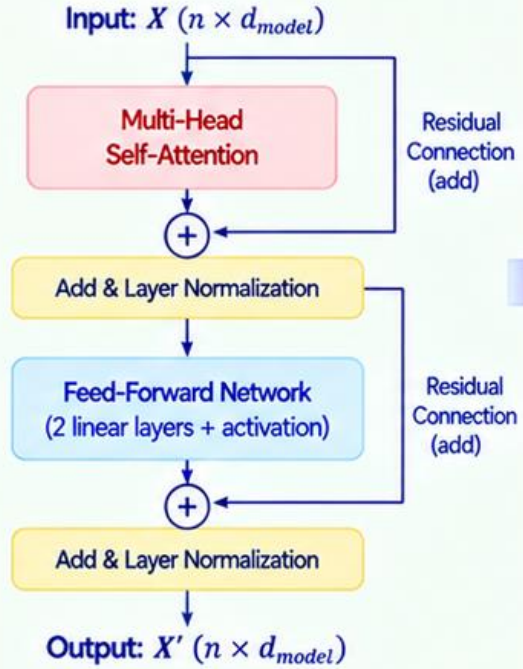
Instead of a single attention head, we use multiple heads in parallel. Each head learns a different set of attention weights and can focus on different aspects of the sentence (e.g. syntax, semantics, coreference).



Output: richer contextual representations by combining information from all heads.

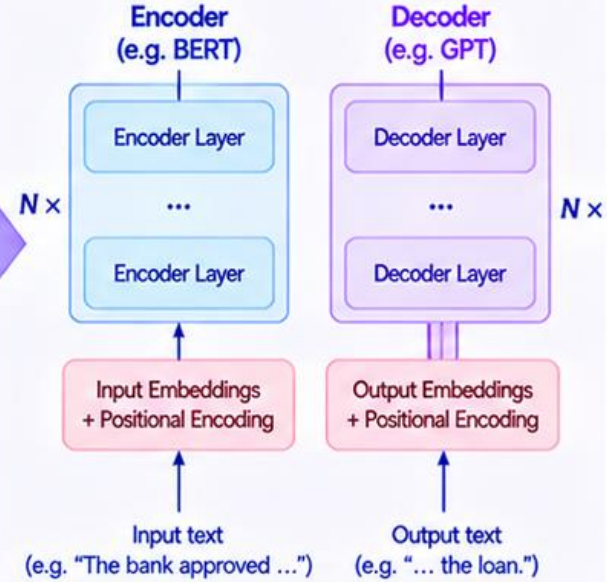
3. Transformer Encoder Layer

In a Transformer, multi-head self-attention is combined with other components to form a complete encoder layer.



4. Full Transformer Architecture

A Transformer consists of a stack of encoder layers (for understanding the input) and decoder layers (for generating output, e.g. in machine translation or text generation).



Key Takeaways

1. Self-attention captures contextual relationships between all words in the sequence.
2. Multi-head attention allows the model to focus on different aspects (syntax, semantics, coreference, etc.).
3. The Transformer combines multi-head self-attention with feed-forward networks, residual connections, layer normalization and positional encoding to build a deep language model.

SIS 3204Y(5)
Natural Language Processing

Week 6 — Final Recap

In this lecture, we explored the concept of attention and how it allows a model to focus on the most relevant words in a sequence.
We built up from the basic idea to self-attention and multi-head attention, and saw how these ideas form the core of the Transformer.

Key Topics Covered

1. **Motivation**
   Why contextual understanding is important in NLP.
2. **The Idea of Attention**
   Focus on relevant words using learned weights.
3. **Query, Key and Value**
   Three representations used to compute attention.
4. **Scaled Dot-Product Attention**
   Compute attention weights using the scaled dot product and softmax.
5. **Self-Attention**
   Each word attends to all other words in the sequence.
6. **Multi-Head Attention**
   Use multiple attention heads in parallel to capture different aspects.
7. **From Self-Attention to Transformers**
   Self-attention combined with other components forms the Transformer.



Main Takeaways

1. Attention allows the model to focus on the most relevant words in a sequence.
2. Self-attention learns relationships between all words, capturing syntax, semantics and long-range dependencies.
3. Multiple attention heads learn different types of relationships in parallel.
4. Self-attention is the core building block of the Transformer, which has achieved state-of-the-art performance in many NLP tasks.

Next Week

Transformers
We will explore the full Transformer architecture, including encoder and decoder layers, positional encoding, and training.

Thank You!

Any Questions?

Keep Learning